Wiki: https://wiki.haskell.org/Algebraic_data_type
Language spec: https://www.haskell.org/onlinereport/haskell2010/haskellch4.html#x10-680004.2

Expressions
Spec: https://www.haskell.org/onlinereport/haskell2010/haskellch3.html#x8-240003.2

```
---- Data constructor
True                    -- nullary data constructor=True of
type=Bool
True :: Bool
False :: Bool
Nothing
Nothing :: Maybe a      -- nullary data constructor=Nothing of
type=Maybe a
Just 1
Just 1 :: Maybe Int     -- unary data constructor=Just 1 of
type=Maybe Int

-- e.g. user defined type
data MyType a = A a | B Int | C
x = A 'a' :: MyType Char
x = B 10 :: MyType a
x = C :: MyType c

-- case: type alias
type MyMaybe a = Maybe a
x = Just 1 :: MyMaybe Int
x :: Maybe Int
x = Nothing :: MyMaybe a
x :: Maybe a


---- case: labeled fields
--
-- i.e. a data with convenient getters + setters
data Point = Point { x:: Int, y::Int }
pt = P { x = 10, y = 20 }
pt = P 10 20                -- same i.e. x=10, y=20

-- partial fields
pt = P { x = 10 }           -- only x specified. not an error, but
warning.
pt :: Point
x pt                        -- 10
y pt                        -- ERROR. y not defined

-- GOTCHA: NOT the same
pt = P { x = 10 }
pt :: Point                  -- pt is a point
pt2 = P 10
pt2 :: Int -> Point          -- NOT a point, but a partially
applied
```

```
-- getter
data Point = Point { x:: Int, y::Int }
pt = Point { x=1, y=10 }
x pt                    -- get field 'x i.e. 10
y pt                    -- 20
Point a b = pt          -- Same without the getters a = 10 b=20

-- setter
-- i.e. create a new data with updated fields
pt2 = pt { y=-1 }                  -- create new copy with optional
field overwrite e.g. pt2 =same as x but with y overwrite
pt2 = Point { x = x pt, y = -1 }   -- same
x pt2                              -- unchanged i.e. 10
x pt2 == x pt1                     -- True
y pt2                              -- changed y=-1


-- pattern matching
pt :: Point -> Int
pt Point {x, y} = x + y                -- bind fields: bind x, y
values
pt Point {x} = x                       -- partial: bind field x
only
pt Point {x = xvalue} = xvalue         -- alias: xvalue for the x
value
pt Point {x = 0} = 0                   -- filter: match x = 0 only
pt p@Point {x} = y p + x               -- bind data: p for the
Point
```

Algebraic data type
Spec: https://www.haskell.org/onlinereport/haskell2010/haskellch4.html#x10-680004.2
GHC extensions: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/types.html

```
----  Basic
--
data MyType = A
data MyType = MyType           -- same name allowed
data MyType =                  -- multiline
  A |
  B
data myType = A | B            -- ERROR. Constructor must begin
with capital.
data MyType = a | B            -- ERROR. Constructor must begin
with capital.

-- case: recursive type allowed
data MyTree a = Node [Tree a] | Leaf a       --

-- e.g.
```

```
Leaf 1 :: MyTree Int
Node [Leaf 1] :: MyTree Int
Node [Leaf 1, Leaf 2]
Node [Leaf 1, Leaf 2, Node [Leaf 3, Leaf 4]]
```

```
-- algebraic sum of data constructors
--
data MyType = A
data MyType = A | B | C                -- nullary constructors
data MyType a b = A a | B a b | C      -- with variables
data MyType a b =              -- combine various
 A |                           -- nullary data constructor
 B b |                         -- unary
 C { v1:: a } |                -- labeled fields
 a :<> b                       -- infix
```

```
-- e.g.
data MyType = A | B | C
x = A :: MyType
x = B :: MyType
x = C :: MyType


-- e.g. combination
data MyType a b =
  A |
  B b |
  C { v1:: a } |
  a :<> b

A :: MyType a b            -- nullary
B 1 :: MyType a Int        -- unary
C { v1=1 } :: MyType a     -- labeled
1 :<> 'a'                  -- infix
```

```
---- n-ary data constructor
--
-- i.e. an object with n-properties
data MyType = A                   -- nullary
data MyType = A Int               -- unary constructor A::Int ->
MyType
data MyType = A Int Char          -- binary A::Int -> Char ->
MyType
data MyType = A (Maybe Int)
data MyType = A Maybe Int         -- ERROR
```

```
-- e.g. nullary
data MyType = A
x = A :: MyType
```

```
-- e.g. binary
data MyType = A Int Char
A :: Int -> Char -> MyType

-- get values
x = A 1 'c' :: MyType
A a b = x                       -- a=1, b='c'
```

```
---- nary + type variables
--
data MyType a = A a                 -- unary type=MyType::* -> *,
unary data constructor=A::a -> MyType a
data MyType a = A a Int             -- unary type=MyType::* -> *,
binary data constructor=A::a -> Int -> MyType a
data MyType a b c = A a b c
data MyType a b c = A               -- dummy variables.
```

```
-- e.g.
data MyType a b = A a b
A::a -> b -> MyType a b
A 1 2 :: MyType Int Int
A 1 'a' :: MyType Int Char

-- dummy
data MyType a b c = A
A :: MyType a b c                    -- dummies can be unspecified
A :: MyType Int Int Char             -- can be anything
```

```
-- strictness
--
-- 2 implications:
-- a) strict field value i.e. evaluate to WHNF, and
-- b) error if a strict field is an error
--
data A a b = A !a !b          -- strict in both a and b
data A a b = A !a b           -- strict in a only
data B f a = B !(f a)         -- strict nary
```

```
data A a b = A !a b

-- equivalent to applying seq
A x y
x `seq` y `seq` A x y         -- same

-- e.g.
A 1 undefined `seq` ()        -- ()            non-strict in b
A undefined 1 `seq` ()        -- undefined     strict in a
```

```
-- type variable kind:
-- i.e. whether the type variables are nullary::*, unary::* -> *,
binary::*->*, etc.
--
-- All are different
data MyType a b c = A a b c
     -- all nullary types
data MyType (a::Type) (b::Type) (c::Type) = A a b c
     -- same
data MyType a b c = A (a b c)
     -- a=binary type
data MyType (a::Type -> Type -> Type) (b::Type) (c::Type) = A a b
c    -- same
data MyType a b c = A a (b c)
   -- b=unary type
data MyType (a::Type) (b::Type->Type) (c::Type) = A a (b c)
    -- same
data MyType a b c = A (a b) c
    -- a=unary type
data MyType (a::Type->Type) (b::Type) (c::Type) = A (a b) c
    -- same
```

```
-- e.g.
data MyType a b c = A a b c
data MyType2 a b c = A2 (a b c)
data MyType3 a b c = A3 a (b c)
data MyType4 a b c = A4 (a b) c

-- kind signature
MyType:: * -> * -> * -> *
MyType2 :: (k1 -> k2 -> *) -> k1 -> k2 -> *
MyType3 :: * -> (k -> *) -> k -> *
MyType4 :: (k -> *) -> k -> * -> *

-- data constructor types
A::a -> b -> c -> MyType a b c        -- all nullary types
A2:: a b c -> MyType2 a b c           -- a=binary type
A2:: a -> b c - MyType2 a b c         -- b=unary type
A3:: a b -> c - MyType2 a b c         -- a=unary type

A 1 2 3 :: MyType Int Int Int     -- OK
A2 1 2 3                          -- ERROR
A3 1 2 3                          -- ERROR
A4 1 2 3                          -- ERROR

A (Left 1)                           -- ERROR
A2 (Left 1) :: MyType2 Either Int a  -- OK
A3 (Left 1)                          -- ERROR
A4 (Left 1)                          -- ERROR
```

```
A 1 (Just 1)                              -- ERROR
A2 1 (Just 1)                             -- ERROR
A3 1 (Just 1) :: MyType3 Int Maybe Int -- OK
A4 1 (Just 1)                             -- ERROR
```

```
-- [GHC extension]: cannot specify context in type definition
--
-- Allowed in the language spec, but disabled by default in GHC
-- Language spec: https://www.haskell.org/onlinereport/haskell2010/haskellch4.html#x10-680004.2
-- GHC: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/datatype_contexts.html
data  (Num a, Show a) => MyType a b = A a | B b           -- ERROR
```

```
-- instead of hard-coding in the type, specify per value
data MyType a b = A a | B b
f :: (Num a, Show a) => MyType a b -> MyType a b        -- OK
f x = id
```

```
---- format: label fields
--
-- i.e. like OOP object properties + getters
data MyType = A { v1::Int }
data MyType = A { v1::Int, v2::Char }        -- multiple
data MyType a b = A { v1::a, v2::b }         -- with variables
data MyType a b = A { v1::a b }              -- composed type. No need for ()
data MyType a b = A { v1::(a b) }            -- same
```

```
-- e.g.
data MyType a b = A { v1::a, v2::b }
x = A { v1=1,v2='a' }
x = A 1 'a'              -- same
x = A { v1=1 }           -- partial
x = A { v2='a' }         -- partial

-- getter
x = A { v1=1,v2='a' }
v1 x            -- 1
v2 x            -- 'a'
A a b = x       -- same without getters  a=1 b='a'

-- e.g. nested type
data MyType a b = A { v1::(a b) }
A (Just 1) :: MyType Maybe Int
A { v1=Just 1 }                      -- same
A 1 2                                -- ERROR
```

```
A { v1=1 2 }                          -- ERROR
```

```
---- format: infix type / infix data constructor
--
-- infix type constructor
-- i.e. a type with infix signature
-- e.g. (->) is an infix type
--    id :: a -> a
data a <> b = A a b
data a <> = A                -- ERROR. Must have exactly 2
variables
data a <> b c = A a b        -- ERROR
data Char <> Int = A         -- ERROR. No concrete type.
data a <z> b = A a b         -- ERROR. Only symbols in operator
name.
```

```
-- -> is a builtin infix type
id :: a -> a

-- e.g. user defined
data a <> b = A a b
A 1 'c' :: Int <> Char
```

```
-- infix data constructor
--
-- i.e. a data with infix constructor
-- e.g. (:) is an infix data constructor
--   1:[2,3] = [1,2,3]
-- MUST begin with ':' symbol
data MyType = Int :<> Char       --
data MyType = Int <> Char        -- ERROR. Must begin with ':'
data MyType a b = a :<> b        -- with type variables
```

```
-- e.g.
data MyType a b = a :<> b
(:<>) :: a -> b -> MyType a b
1 :<> [1] :: MyType Int [Int]
```

```
-- infix type + data constructor
-- Not common? But legal.
data a <> b = a :<> b           -- type=a <> b  data
constructor=a :<> b
```

```
-- e.g.
data a <> b = a :<> b
(:<>) :: a -> b -> a <> b           -- data constructor :<>
1 :<> [1] :: Int <> [Int]           -- type=[Int] <> [Int]
```

Type synonym
Spec: https://www.haskell.org/onlinereport/haskell2010/haskellch4.html#x10-680004.2

Type synonym
i.e. type **alias**.
i.e. can apply same methods
e.g.

```
type Height = Int
1 :: Int
1 :: Height              -- same
(1::Height) + (1::Int)   -- OK
```

```
---- Basics
--
type Width = Int
```

```
-- e.g.
x = 1 :: Width
```

```
type X a = [a]        -- with variables
```

```
-- e.g.
[1] :: X Int
['c'] :: X Char
```

```
type X a b = a b
```

```
[1] :: X [] Int
Just 'a' :: X Maybe Char
```

```
-- GOTCHA: arity
-- NOT the same
type X0 = Either            -- arity 0     can partially apply
Either
type X1 a = Either a        -- arity 1     can only partially apply
Either a
type X2 a b = Either a b    -- arity 2     can NOT partially apply
```

```
X0             -- Either OK       i.e. partial application
X0 Int         -- Either Int
X0 Int Bool    -- Either Int Bool
X1             -- ERROR
X1 Int         -- Either Int
X1 Int Bool    -- Either Int Bool
```

```
X2              -- ERROR
X2 Int          -- ERROR
X2 Int Bool     -- Either Int Bool
```

newtype (renamed type)
Spec: https://www.haskell.org/onlinereport/haskell2010/haskellch4.html#x10-680004.2

newtype (renamed type)
Like a middle ground between algebraic data type and type alias:
- like type alias, exactly **one value** only.
- like algebraic data type, has a **data constructor**.
- **strict** by default

```
---- Basics
--
newtype MyType = A Int

-- instance
A 10

newtype MyType = MyType Int                         -- OK. Same name
as type.
newtype MyType a b = A (a -> b)                     -- with variables
newtype MyType (a::Type -> Type) b = A (a b)        -- OK. a=unary
type variable
newtype MyType = A Int Int                          -- ERROR. Can only
have one value
newtype MyType = A Int | B Int                      -- ERROR
newtype MyType = A Int deriving(Eq,Ord)             -- deriving
newtype MyType a = A !a                             -- ERROR. strict
by default.

-- with fields
newtype MyType = A {getA::Int}
newtype MyType a b= A {getA::a->b}
```

==========

GHC Extensions

GHC extension: Generalized algebraic data types (GADT)
GADT syntax: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/gadt_syntax.html#gadt-style
GADT: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/gadt.html

Generalized algebraic data types (GADT)
Generalize by separating **data constructor** from **type**.
i.e. define **type signatur**e for each data constructor

usage: pattern matching

```
{-# LANGUAGE GADTs #-}
{-# LANGUAGE OverloadedStrings #-}

import GHC.Exts
import Data.Char

-- without GADT
data Dyn0 a = S0 String |  C0 Char deriving(Show, Eq)

-- with GADT
data Dyn a where
  S :: IsString s => {getS :: s} -> Dyn s
  C :: {getC :: Char} -> Dyn Char
  N :: Num a => {getN :: a} -> Dyn a

-- e.g.
-- with
isEmpty0 :: Eq a => Dyn0 a -> Bool
isEmpty0 (S v) = v == fromString ""       --  ERROR "Couldn't match
Dyn0 a with actual type Dyn0 a0"

-- with GADT
isEmpty :: Eq a => Dyn a -> Bool
isEmpty (S v) = v == fromString ""        -- v :: String
isEmpty (C c) = ord c == 0
isEmpty (N v) = v == 0
```

```
---- Basics
--
-- sum
{-# LANGUAGE GADTs #-}
data MyType where
  A :: MyType
  B :: MyType
  C :: MyType
```

```
data MyType = A | B | C
```

```
-- product
data MyType a b where
  MkMyType :: a -> b -> Int -> Char -> MyType a b
```

```
data MyType a b = MkMyType a b Int String
```

```
-- field
data MyType where
   MkMyType :: { v1 :: Int, v2 :: Int } -> MyType
```

```
data MyType = MkMyType { v1 :: Int, v2 :: Int ]
```

```
-- existential
data MyException where
  MkMyException :: forall a. a -> MyException

data MyException = forall a. MkMyException a
```

GHC Extension: Existentially qualified data constructors
Ref: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/existential_quantification.html#extension-ExistentialQuantification

Existential qualification
A type defined by **existence**
x is type T <=> there exists x such that Q x
In particular, allow **grouping** different types under a single type

e.g. Exceptions

```
-- Control.Exception
data SomeException = forall e. Exception e => SomeException e
data Handler a = forall e. Exception e => Handler (e -> IO a)

x = SomeException DivideByZero        -- DivideByZero ::
ArithException
x = SomeException StackOverflow       -- StackOverflow ::
AsyncException
```

```
---- Basics
{-# LANGUAGE ExistentialQuantification #-}
data MyType = forall a. MyType a
data MyType = forall a. Eq a => MyType a            -- with context
data MyType = forall a. (Eq a, Show a) => MyType a  -- multiple
data MyType = (Eq a, Show a) => MyType a            -- ERROR. Need
explicit forall

-- with explicit kind signatures
{-# LANGUAGE KindSignatures #-}
import Data
data MyType = forall (a :: Type). MyType a
data MyType = forall (m :: Type -> Type) (a :: Type). MyType (m a)

-- e.g.
data MyType = forall a. Eq a => MyType a
x :: MyType
x = MyType (1 :: Int)        -- OK. MyType with Int
x = MyType True              -- OK. MyType with True
x = MyType id                -- ERROR. function is not an Eq
```

usage: container for heterogenous types
e.g. container for any Show values

```haskell
{-# LANGUAGE ExistentialQuantification #-}
import Data.List (intercalate)
import Data.Foldable

data Showable = forall a. Show a => MkShowable a
instance Show Showable where
  show (MkShowable x) = show x

-- e.g. list of Show values
list = [
    MkShowable 1,
    MkShowable True,
    MkShowable $ Just 1,
    MkShowable "A"
  ]
io :: IO ()
io = traverse_ print list                   -- e.g. print each item
joined :: String
joined = intercalate "," (show <$> list)    -- e.g. join into
single string "1,True,Just 1,A"
```